FEU INSTITUTE OF TECHNOLOGY

COLLEGE OF COMPUTER STUDIES AND MULTIMEDIA ARTS

CS0065
(INTELLIGENT SYSTEMS)

TECHNICAL ASSESSMENT

3

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Student Name	*Calpo, Santino Luis Javier P.*
Section:	*BSCSSE – AN43*
Professor:	*Ms Crisola Tan*
Date:	*11/1/2026*

I. BACKGROUND INFORMATION

Case-Based Reasoning (CBR) *is an intelligent problem-solving approach used in* **Knowledge Representation and Reasoning (KRR)**. *Instead of relying solely on predefined rules, CBR systems solve new problems by referring to past experiences, also known as cases. These cases contain previous problems and their successful solutions. When a new situation arises, the system identifies the most similar past case, adapts the old solution to fit the new context, and applies it. Afterward, the new case is saved, allowing the system to learn and continuously improve.*

CBR is widely used in various fields, such as disaster response, legal assistance, and intelligent tutoring, because it mimics how humans naturally make decisions. For example, emergency responders rely on past typhoon data to plan evacuations; lawyers examine previous legal cases to build arguments; and tutoring systems look at prior student mistakes to provide customized guidance. In all these examples, past knowledge becomes the foundation for solving future problems more effectively.

This activity allows students to experience how CBR works by developing a simple, functional CBR program based on real-world scenarios. Through pair programming, students will learn how to represent cases, implement similarity assessment, adapt solutions, and store new learning experiences—reflecting the complete CBR cycle. This hands-on approach strengthens understanding of how intelligent systems use experiential knowledge to support reasoning, decision-making, and learning.

II. TECHNICAL ASSESSMENT

CASE-BASED REASONING (CBR) SYSTEM DEVELOPMENT

Objective

At the end of this laboratory activity, students are expected to:

1. **Understand and apply the Case-Based Reasoning (CBR) cycle**, *including retrieval, reuse, revision, and retention.*
2. **Represent cases effectively** *using appropriate data structures such as dictionaries, arrays, JSON objects, or classes.*
3. **Implement a similarity assessment method** *to identify relevant past cases based on defined attributes.*
4. **Adapt retrieved solutions** *to meet the needs of a new problem scenario based on the chosen case study.*
5. **Demonstrate learning in CBR** *by saving the newly solved case into the system's case base.*
6. **Develop collaboration skills** *through pair programming and joint problem analysis.*
7. **Apply knowledge representation and reasoning concepts** *to real-world problems in disaster response, legal reasoning, or intelligent tutoring.*

Materials Needed

1. Hardware

- *A laptop or desktop computer*
- *Stable internet connection (for research, documentation, submission)*

2. Software

A programming environment:

- *Python (recommended: IDLE, VS Code, Jupyter Notebook, PyCharm)*
- *Any text editor (Notepad++, Sublime, VS Code) if needed*

3. File Formats & Tools

Word processor for documentation:

- *MS Word, Google Docs, or LibreOffice*
- *Screenshot tool (e.g., Snipping Tool, Lightshot, Print Screen)*

4. Reference Materials

Lecture notes on:

- *Knowledge Representation & Reasoning*
- *Case-Based Reasoning cycle*
- *Similarity measures*

Provided case studies (Disaster DSS, Legal DSS, Intelligent Tutoring System)

5. Pair Collaboration Tools (optional but useful)

- *Google Drive for shared documentation*
- *GitHub/GitLab for code collaboration*

General Instructions

1. *This* **Technical Assessment (TA3)** *must be completed by Pair.*
2. *Please ensure that you are grouped as pairs in the People tab.*
3. *Select only one case study from the list below.*
4. *Include clear comments in your code to describe its logic.*
5. *Each program should display the correct input, process, and output.*
6. *Ensure that your script displays clear and readable outputs.*
7. *Compile all Python codes and corresponding outputs into one PDF file.*
8. *Save it as* **TA3_lastname1_lastname2.pdf**
   (Example: TA2_Patilano_Loverez.pdf)
9. *Submit the* **python file** *and* **PDF** *file.*

📌 *Step-by-Step Instructions*

1. Form a pair

Choose a partner. Each pair will work collaboratively on research, coding, debugging, and presenting your results.

2. Choose ONE (1) case study

Select only one case study from the list below. Your program must be based on the scenario and logic of the chosen system:

Case Study Options:

1. *Disaster Response Decision Support System (DSS)*
2. *AI-Based Legal Decision Support System*

3. *Intelligent Tutoring System (ITS)*

3. Define your case base

Create at least **3–5 past cases** *stored as dictionaries, JSON objects, arrays, or any structure.*

Example structure:

```
{
  "problem": {...},
  "solution": {...}
}
```

4. Implement similarity assessment

Your program must include a way to measure similarity between the new case and past cases. Examples:

- *Attribute matching*
- *Weighted scoring*
- *Simple rule-based comparison*

Output should show: (sample)

✓ *Similarity score or ranking*

✓ *The most relevant past case retrieved*

5. Adapt the solution

Modify the retrieved solution so it fits the new problem scenario.
This part should reflect the logic of your chosen case study:

Examples:

- **DSS:** *Adjust evacuation plans based on population size*
- **Legal DSS:** *Modify arguments due to new digital evidence*
- **ITS:** *Give personalized feedback based on student errors*

6. Demonstrate learning (retain stage)

After generating a final solution, store the new case into your case base.

Your program must show:

✓ *"New case saved" or similar confirmation*

✓ *Updated case list*

7. Display clear program outputs

Your output must include:

- *New problem description*
- *Retrieved similar case*
- *Adapted final solution*
- *Updated case base*

8. Submission Requirements

Submit the following:

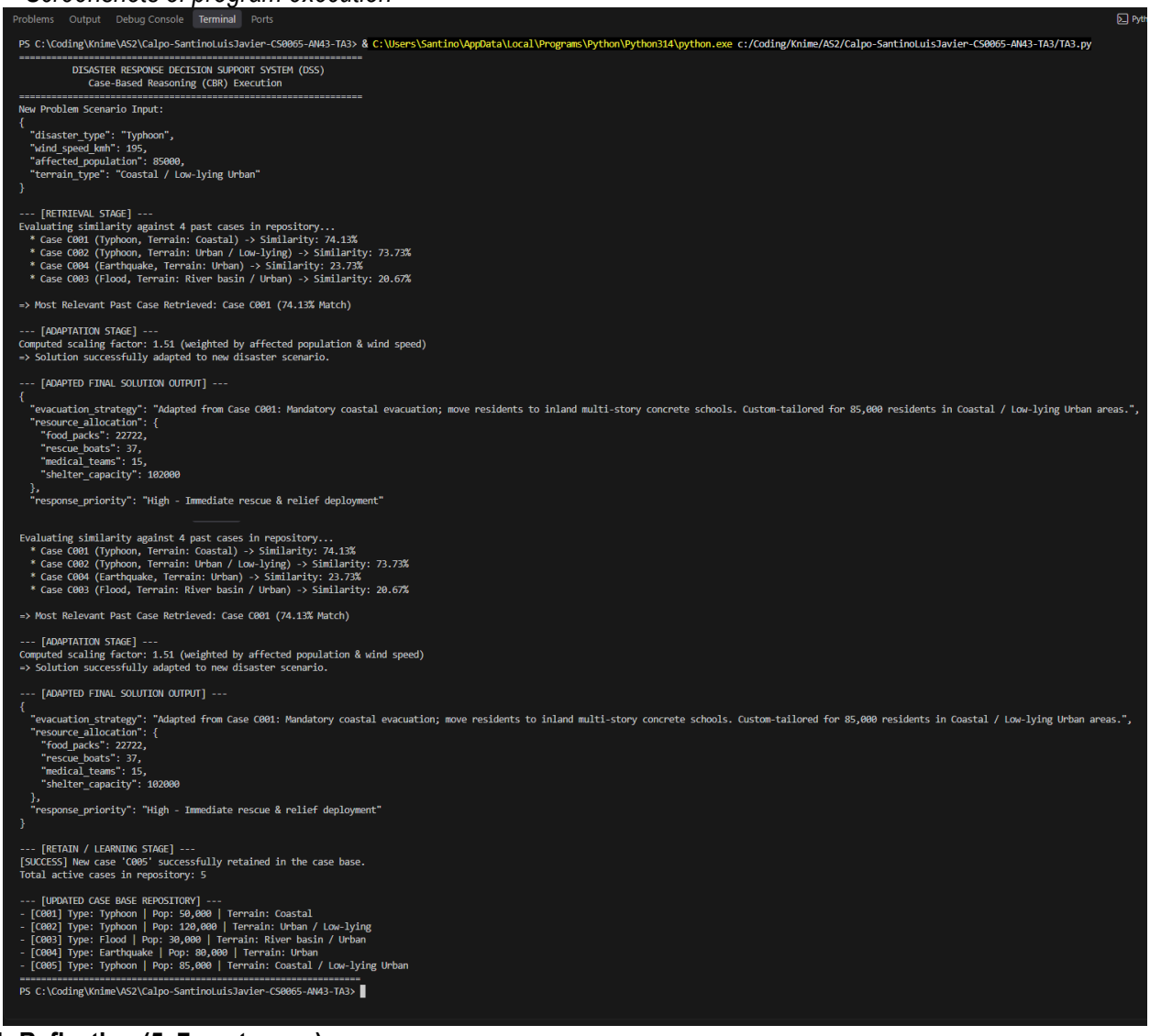
A. Program File

- *Python*

B. Short Documentation (1–2 pages)

Include:

- *Title of chosen case study*
- *Description of your case base*
- *Explanation of similarity method*
- *Adaptation logic*
- *Screenshots of program execution*



C. Pair Reflection (5–7 sentences)

Discuss:

- *What each partner contributed*

- *What challenges you encountered*
- *What you learned about CBR*

Case Studies:

Case Study 1: Disaster Response Decision Support System

A CBR-based DSS helps local government units respond to natural disasters.

- *Past cases include typhoons, floods, and earthquakes.*
- *New event: Typhoon with expected flooding in urban areas.*
- *System retrieves similar events: Typhoon Ursula (2019) and Typhoon Odette (2021).*
- *It adapts recommended responses such as evacuation strategy, resource allocation, and shelter capacity.*
- *After execution, the system stores new outcomes for improved future reasoning.*

Learning Outcome:

CBR helps governments respond faster by using experiential knowledge rather than rebuilding decisions from scratch.

Case Study 2: AI-Based Legal Decision Support System

A legal CBR tool assists lawyers by retrieving previous court cases similar to the current legal issue.

Scenario:

Client is charged with "theft involving electronic transfer." System retrieves past cases involving:

- *online theft*
- *digital fraud*
- *unauthorized bank transfers*

Adaptation:

The reasoning engine adjusts argument patterns to match the new technical context.

Learning:

After final verdict, the case is stored—enhancing the system's knowledge for future cybercrime-related offenses.

Case Study 3: Intelligent Tutoring System

A CBR system identifies the learning struggles of students.
Student submits a math solution with repeated errors.
System retrieves past students with similar error patterns (e.g., misusing formulas).
It adapts feedback and personalized lessons based on successful interventions.

Learning:

System improves by retaining new student profiles and outcomes.



Your Code - Laboratory Tasks

```
"""
================================================================================
```

```
Technical Assessment 3: Case-Based Reasoning (CBR) System Development
Course: CS0065 - Intelligent Systems
Institution: FEU Institute of Technology (FEU Alabang)
Case Study: Disaster Response Decision Support System (DSS)
Developer: Santino Luis Javier P. Calpo
================================================================================
"""

import json

class DisasterCBRSystem:
    def __init__(self):
        # Initial Case Base: 4 historical disaster cases stored as structured
dictionaries
        self.case_base = [
            {
                "case_id": "C001",
                "problem": {
                    "disaster_type": "Typhoon",
                    "wind_speed_kmh": 180,
                    "affected_population": 50000,
                    "terrain_type": "Coastal"
                },
                "solution": {
                    "evacuation_strategy": "Mandatory coastal evacuation; move
residents to inland multi-story concrete schools.",
                    "resource_allocation": {
                        "food_packs": 15000,
                        "rescue_boats": 25,
                        "medical_teams": 10,
                        "shelter_capacity": 60000
                    },
                    "response_priority": "High - Immediate rescue & relief
deployment"
                }
            },
            {
                "case_id": "C002",
                "problem": {
                    "disaster_type": "Typhoon",
                    "wind_speed_kmh": 215,
                    "affected_population": 120000,
```

```
                    "terrain_type": "Urban / Low-lying"
                },
                "solution": {
                    "evacuation_strategy": "Urgent vertical and horizontal evacuation from flood-prone urban zones to designated gymnasiums and higher ground.",
                    "resource_allocation": {
                        "food_packs": 40000,
                        "rescue_boats": 60,
                        "medical_teams": 25,
                        "shelter_capacity": 130000
                    },
                    "response_priority": "Critical - Multi-agency mobilization"
                }
            },
            {
                "case_id": "C003",
                "problem": {
                    "disaster_type": "Flood",
                    "wind_speed_kmh": 45,
                    "affected_population": 30000,
                    "terrain_type": "River basin / Urban"
                },
                "solution": {
                    "evacuation_strategy": "Controlled relocation of riverside communities to municipal flood shelters.",
                    "resource_allocation": {
                        "food_packs": 10000,
                        "rescue_boats": 20,
                        "medical_teams": 6,
                        "shelter_capacity": 35000
                    },
                    "response_priority": "Moderate - Sandbagging and localized rescue"
                }
            },
            {
                "case_id": "C004",
                "problem": {
                    "disaster_type": "Earthquake",
                    "wind_speed_kmh": 0,
                    "affected_population": 80000,
```

```
                    "terrain_type": "Urban"
                },
                "solution": {
                    "evacuation_strategy": "Open-area assembly in parks and sports
complexes; structural damage inspection.",
                    "resource_allocation": {
                        "food_packs": 25000,
                        "rescue_boats": 0,
                        "medical_teams": 30,
                        "shelter_capacity": 90000
                    },
                    "response_priority": "Critical - Search and rescue, medical
triage"
                }
            }
        ]

    def calculate_similarity(self, new_problem, past_problem):
        """
        Calculates similarity score (0.0 to 1.0) using weighted multi-attribute
matching:
        - Disaster Type match: 40% weight
        - Terrain Type match: 20% weight
        - Wind Speed difference: 20% weight (normalized)
        - Affected Population difference: 20% weight (normalized)
        """
        score = 0.0

        # 1. Disaster Type (Exact match = 0.40)
        if                new_problem["disaster_type"].lower()                ==
past_problem["disaster_type"].lower():
            score += 0.40

        # 2. Terrain Type (Exact match = 0.20, partial keyword match = 0.15)
        new_terrain = new_problem["terrain_type"].lower()
        past_terrain = past_problem["terrain_type"].lower()
        if new_terrain == past_terrain:
            score += 0.20
        elif  any(term  in  past_terrain  for  term  in  new_terrain.split("/")  if
len(term.strip()) > 3):
            score += 0.15
```

```
        # 3. Wind Speed Similarity (Normalized absolute difference, max assumed
250 km/h)
        wind_diff          =          abs(new_problem["wind_speed_kmh"]          -
past_problem["wind_speed_kmh"])
        wind_sim = max(0.0, 1.0 - (wind_diff / 250.0))
        score += 0.20 * wind_sim

        # 4. Affected Population Similarity (Normalized absolute difference, max
assumed 150,000)
        pop_diff         =         abs(new_problem["affected_population"]         -
past_problem["affected_population"])
        pop_sim = max(0.0, 1.0 - (pop_diff / 150000.0))
        score += 0.20 * pop_sim

        return round(score, 4)

    def retrieve(self, new_problem):
        """
        Retrieval Stage: Evaluates all past cases and ranks them by similarity
score.
        """
        similarities = []
        for case in self.case_base:
            sim_score = self.calculate_similarity(new_problem, case["problem"])
            similarities.append((sim_score, case))

        # Sort by similarity descending
        similarities.sort(key=lambda x: x[0], reverse=True)
        best_score, best_case = similarities[0]

        print("\n--- [RETRIEVAL STAGE] ---")
        print(f"Evaluating similarity against {len(self.case_base)} past cases in
repository...")
        for score, c in similarities:
            print(f"    *  Case  {c['case_id']}  ({c['problem']['disaster_type']},
Terrain: {c['problem']['terrain_type']}) -> Similarity: {score * 100:.2f}%")
        print(f"\n=>     Most     Relevant     Past     Case     Retrieved:     Case
{best_case['case_id']} ({best_score * 100:.2f}% Match)")

        return best_case, best_score
```

```
    def adapt(self, new_problem, retrieved_case):
        """
        Adaptation Stage: Adjusts resource allocation and evacuation plans
        proportionally based on population ratio and wind intensity.
        """
        print("\n--- [ADAPTATION STAGE] ---")
        old_prob = retrieved_case["problem"]
        old_sol = retrieved_case["solution"]

        # Compute scaling ratios
        pop_ratio = new_problem["affected_population"] / max(1,
old_prob["affected_population"])
        wind_ratio = (new_problem["wind_speed_kmh"] + 1) / max(1,
(old_prob["wind_speed_kmh"] + 1))

        scaling_factor = (pop_ratio * 0.7) + (wind_ratio * 0.3)
        print(f"Computed scaling factor: {scaling_factor:.2f} (weighted by
affected population & wind speed)")

        # Generate adapted solution dictionary
        adapted_solution = {
            "evacuation_strategy": f"Adapted from Case
{retrieved_case['case_id']}: {old_sol['evacuation_strategy']} Custom-tailored for
{new_problem['affected_population']:,} residents in {new_problem['terrain_type']}
areas.",
            "resource_allocation": {
                "food_packs": int(old_sol["resource_allocation"]["food_packs"] *
scaling_factor),
                "rescue_boats":
int(old_sol["resource_allocation"]["rescue_boats"] * scaling_factor),
                "medical_teams":
int(old_sol["resource_allocation"]["medical_teams"] * scaling_factor),
                "shelter_capacity":
int(old_sol["resource_allocation"]["shelter_capacity"] * pop_ratio)
            },
            "response_priority": old_sol["response_priority"]
        }

        print("=> Solution successfully adapted to new disaster scenario.")
        return adapted_solution
```

```
    def retain(self, new_problem, adapted_solution):
        """
        Retain (Learning) Stage: Stores the newly solved case into the case base.
        """
        print("\n--- [RETAIN / LEARNING STAGE] ---")
        new_case_id = f"C00{len(self.case_base) + 1}"
        new_case = {
            "case_id": new_case_id,
            "problem": new_problem,
            "solution": adapted_solution
        }
        self.case_base.append(new_case)
        print(f"[SUCCESS] New case '{new_case_id}' successfully retained in the
case base.")
        print(f"Total active cases in repository: {len(self.case_base)}")
        return new_case_id

    def run_cbr(self, new_problem):
        print("================================================================")
        print("          DISASTER RESPONSE DECISION SUPPORT SYSTEM (DSS)")
        print("             Case-Based Reasoning (CBR) Execution")
        print("================================================================")
        print("New Problem Scenario Input:")
        print(json.dumps(new_problem, indent=2))

        # Execute CBR Cycle
        retrieved_case, best_score = self.retrieve(new_problem)
        adapted_sol = self.adapt(new_problem, retrieved_case)

        print("\n--- [ADAPTED FINAL SOLUTION OUTPUT] ---")
        print(json.dumps(adapted_sol, indent=2))

        self.retain(new_problem, adapted_sol)

        print("\n--- [UPDATED CASE BASE REPOSITORY] ---")
        for c in self.case_base:
            print(f"-  [{c['case_id']}]  Type:  {c['problem']['disaster_type']}  |
Pop:          {c['problem']['affected_population']:,}         |          Terrain:
{c['problem']['terrain_type']}")
        print("================================================================")
```

```
if __name__ == "__main__":
    # Define a new incoming disaster scenario
    incoming_disaster = {
        "disaster_type": "Typhoon",
        "wind_speed_kmh": 195,
        "affected_population": 85000,
        "terrain_type": "Coastal / Low-lying Urban"
    }

    dss = DisasterCBRSystem()
    dss.run_cbr(incoming_disaster)
```